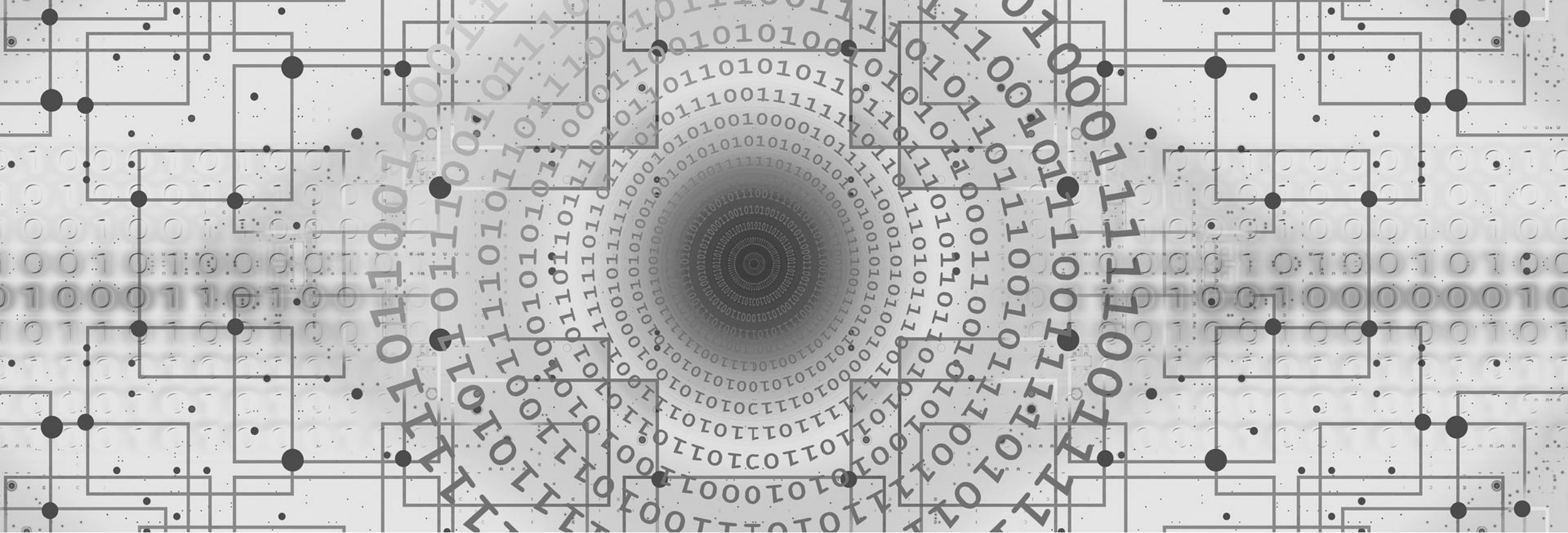




Homework-5c: ECC Encoder and ECC Decoder (SECDED)

ECE-111 Advanced Digital Design Project

Presentation Outline

- ❑ **Let's first understand data corruption due to bit flips and its :**
 - causes, consequences, and prevention techniques

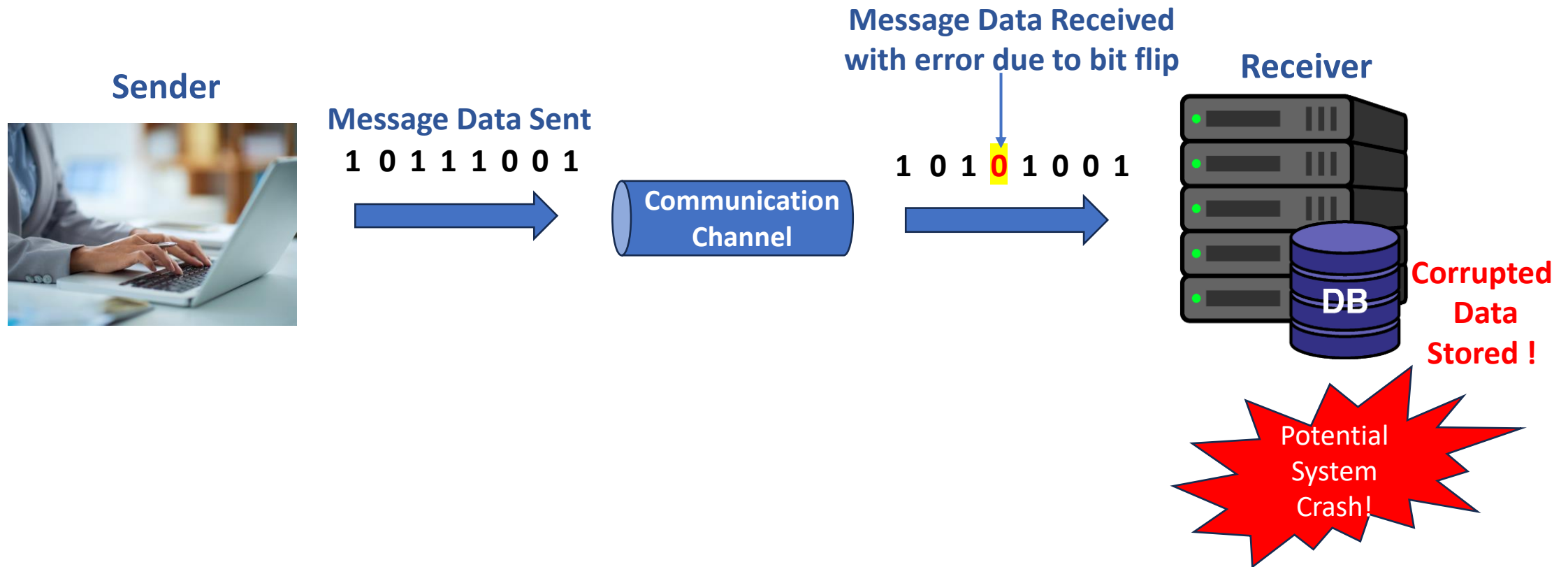
- ❑ **Overview of parity bit as this would be used for error detection**
 - ODD and EVEN parity
 - Limitations of parity bit

- ❑ **Hamming Error Correcting Code Algorithm for encoder and decoder to address data corruption issue due to single bit flip**
 - Single Error Correct, Double Error Detect (SECDDED) Algorithm
 - Examples of encoder and decoder

- ❑ **Homework requirements for SECDDED ECC Encoder and Decoder RTL model**

Data Corruption Can Happen During Transmission !

- ❑ Data can get corrupted when transmitted from sender to receiver over any communication channel
 - If receiver system is incapable of detecting errors when message is received, this may lead to system crash or cause unreliable operation of the system!



Some of the causes of a bit flip

❑ Electromagnetic interference (EMI) :

- EMI is the interference of external electromagnetic fields with the operation of electronic devices
- EMI can cause bit flips by inducing currents in the circuits or altering the voltage levels, leading to incorrect data storage or transmission.
- EMI can be caused by various sources, such as radio frequency interference, power surges, and lightning strikes.

❑ Manufacturing defects :

- Memory chips can cause bit flips due to the presence of impurities, defects in the oxide layer, or incomplete wiring.
- These defects can result from various factors, such as dust, contamination, and insufficient quality control during production

❑ Software Issues :

- Software bugs, glitches or programming errors can cause bit flips
- Software bug may inadvertently overwrite a memory location, causing stored data to change

Some of the causes of a bit flip

❑ Overclocking :

- Increasing the clock speed of a computer CPU or Memory beyond the manufacturer recommendation limit to achieve higher performance can lead to :
 - Overheating, instability, and bit flips

❑ Aging of hardware :

- Over time electronic components of a computing system degrade and become more susceptible to bit flips.
 - degradation can be caused by heat, humidity, voltage fluctuations, leading to reduced performance and reliability

❑ Cosmic rays :

- Cosmic rays are high-energy particles that originate from outer space and can penetrate the earth's atmosphere and affect computing systems.
- When cosmic rays collide with atoms in the atmosphere, they create secondary particles, including neutrons, which can cause bit flips in memory cells

Some of the consequences of a bit flip if undetected !

☐ **Silent data corruption :**

- Bit flip if undetected, can lead to incorrect data storage, transmission or processing, leading to errors or failures in critical systems

☐ **Compute System Crash :**

- Shut down of compute system unexpectedly in middle of ongoing workload execution

☐ **Security vulnerabilities :**

- Hacker can exploit altered data to gain unauthorized access or execute malicious code to bring system down

☐ **Financial losses :**

- Data corruption due to a bit flip in stock market trading system can lead to incorrect trades and financial losses
- Data corruption due to a bit flip in a healthcare system can lead to incorrect diagnoses and treatments of patient, leading to legal implications and loss of reputation.

☐ **Legal implications :**

- Bit flip can have legal implications in cases where the altered data leads to legal disputes or compliance violations

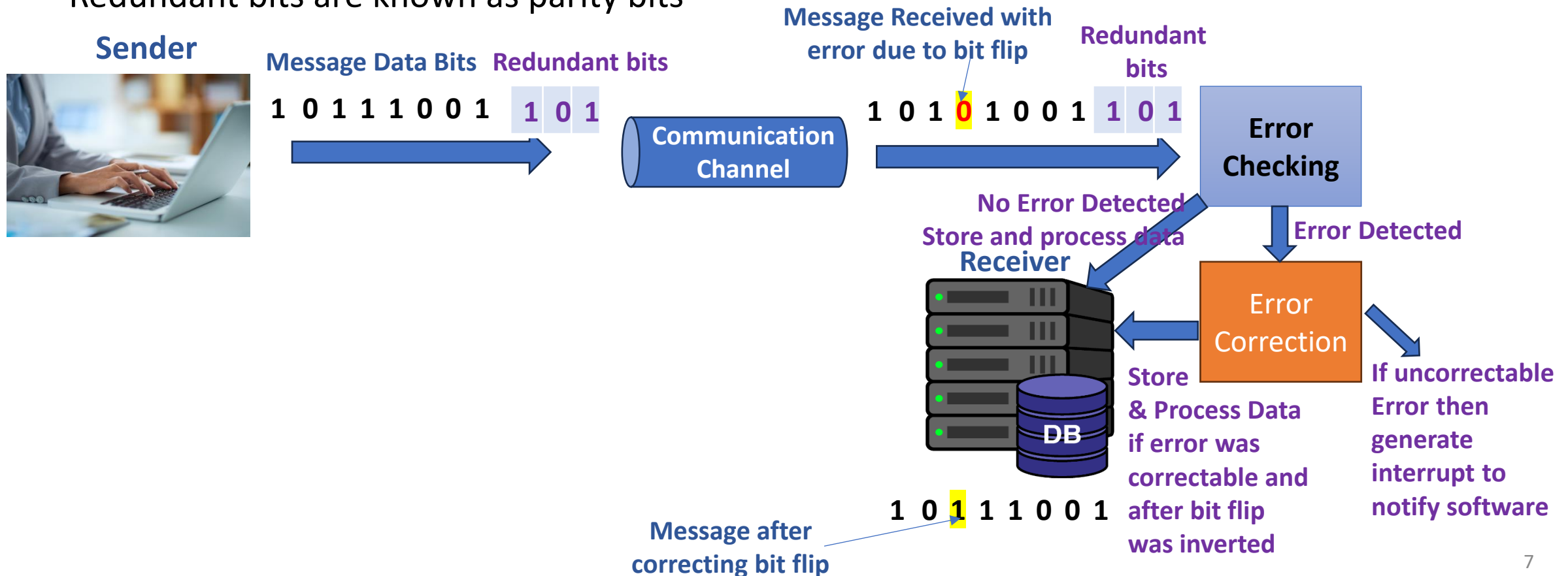
☐ **Human loss :**

- Bit flip in self autonomous vehicles, if undetected can cause vehicle's decision-making system make incorrect decisions and lead to accidents and potential human loss.

Prevention of a bit flip

❑ Error-correcting code (ECC) :

- Error correcting code is a technique that adds redundant bits to memory chips to detect and correct bit flips
- ECC can detect and correct single bit errors and detect double-bit errors, overall improving reliability and accuracy of computing systems
- Redundant bits are known as parity bits



Prevention of a bit flip

❑ Other techniques for prevention of a bit flip :

- Redundancy and backup at hardware and software level :
 - Take a copy of data and store in multiple locations. Compare redundant data copy and pick the ones which are matching
- Shielding and grounding electronics components from electromagnetic fields and reduced risk of EMI caused bit flips
- Quality assurance and rigorous testing of computing systems hardware and software to identify bit flip related potential issues and fix it before it they can occur
- Regular maintenance of hardware including cooling, cleaning and updating software
- Replacement of hardware components that have reached their end of the life

What is an ODD and EVEN Parity ?

❑ A **parity bit**, is a bit added to a binary code to ensure that the total number of 1-bits in the string is either **even** or **odd**.

❑ **There are two types of parity :**

- **Even parity :** (goal is to make bit stream to have even number of 1's)
 - Count of 1-bits in a bitstream is already even then parity bit with value 0 is added
 - Count of 1-bits is odd then parity bit with value 1 is added to make it even number of 1's
- **Odd parity :** (goal is to make bit stream to have odd number of 1's)
 - Count of 1-bits in a bitstream is already odd then parity bit with value 0 is added
 - Count of 1-bits is even then parity bit with value 1 is added to make it odd number of 1's

Example of Odd and Even Parity

7 bits of data	(count of 1-bits)	8 bits including parity	
		even	odd
0000000	0	00000000	10000000
1010001	3	11010001	01010001
1101001	4	01101001	11101001
1111111	7	11111111	01111111

Even Parity generation for 7 bit input:

$$P = A(0) \wedge A(1) \wedge A(2) \wedge A(3) \wedge A(4) \wedge A(5) \wedge A(6)$$

Odd Parity generation for 7 bit input:

$$P = A(0) \wedge A(1) \wedge A(2) \wedge A(3) \wedge A(4) \wedge A(5) \sim A(6)$$

Reference:

https://en.wikipedia.org/wiki/Parity_bit

1-bit error detection using parity, but parity cannot correct error !

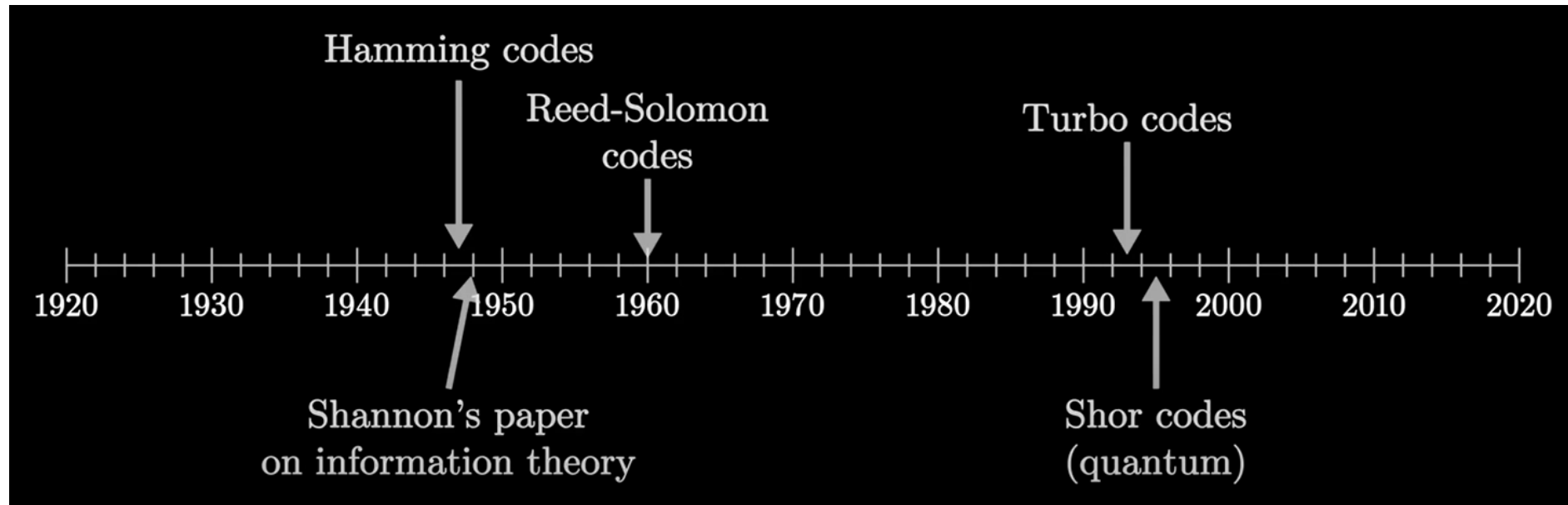
- ❑ If sender transmits data packet with odd number of 1's (including the parity bit) and receiver receives data packet with even number of 1's, then :
 - Receiver will calculate incorrect parity resulting in parity error indicating bit flip in transmission
 - In case of parity error, data must be discarded entirely, and re-transmitted
 - Parity uses only a single bit and requires only a smaller number of **XOR** gates to generate
 - Parity bit is only suitable for detecting errors and **it cannot correct errors** as there is no way to determine which bit is corrupted
 - Hence techniques like **Error Correction Codes (ECC)** are required to correct errors such as Hamming Code, Reed-Solomon codes, etc.

Example of Detecting Error in Transmission using Parity

Type of bit parity error	Failed transmission scenario
Even parity Error in the second bit	Alice wants to transmit: 1001 Alice computes parity bit value: $1 \oplus 0 \oplus 0 \oplus 1 = 0$ Alice adds parity bit and sends: 10010 ...TRANSMISSION ERROR... Bob receives: 11010 Bob computes overall parity: $1 \oplus 1 \oplus 0 \oplus 1 \oplus 0 = 1$ Bob reports incorrect transmission after observing unexpected odd result.

Error Correction Codes

- ❑ There are many error correction code techniques which are deployed in digital communication systems. See timeline of some of these below :



Picture Source : 3Blue1Brown, How to send a self-correction message, <https://www.youtube.com/watch?v=X8jsijhIIIA>

- ❑ Further we will discuss Hamming Codes for Error Detection and Correction !

Applications of ECC

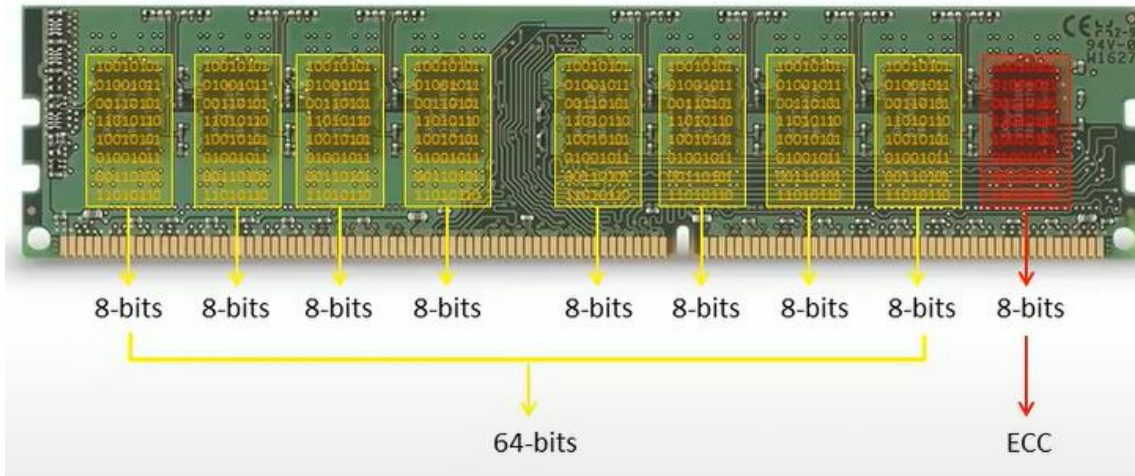
❑ **ECC is used in hardware applications in which data corruption due to bit flips is unacceptable. Some of these applications are :**

- Embedded Processor
- Computer Memory
- Database File Server Systems
- Satellites communications
- Modems
- Autonomous Automobile ADAS systems
- Plasma CAM
- Shielding wire

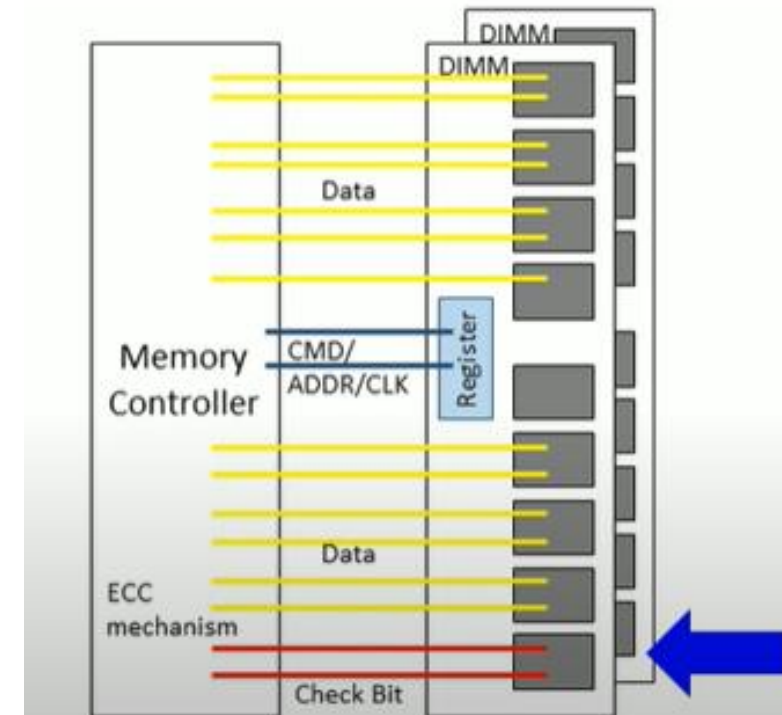
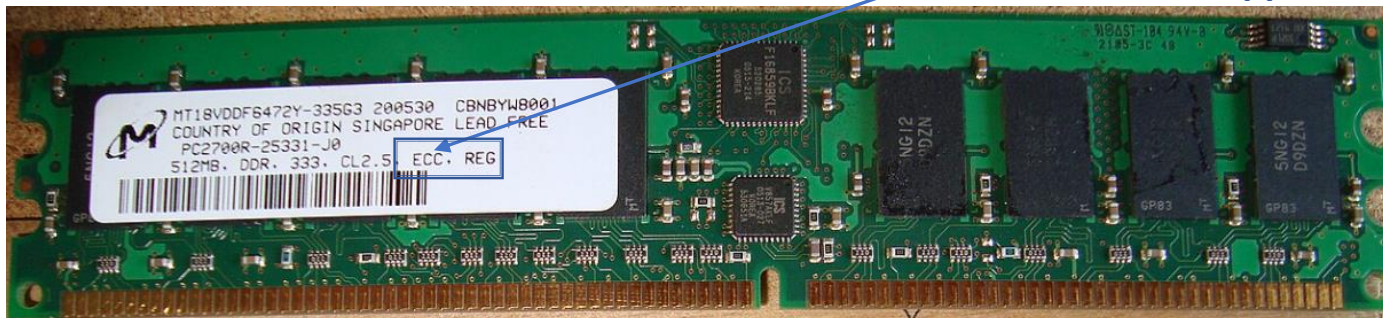
ECC in DRAM

- ❑ Electrical or magnetic interference inside a computer system can cause a single bit of dynamic random-access memory (DRAM) to spontaneously flip to the opposite state
 - ECC bits are typically used to correct the noise while reading the data from the DRAM

DRAM With ECC Bits For Single Bit Error Detection and Correction
(Example : For each 64-bit DWORD there are 8-bits of ECC added)



DRAM with ECC Support



DRAM
Memory
Controller
Upon reading
the data
performs ECC
Checking to
detect errors

Hamming Code

- ❑ **Hamming code is an error correcting system that can detect errors and correct errors when data is stored or transmitted from the sender to the receiver.**
 - It can detect up to two simultaneous bit errors and correct single bit errors
 - Hence this **ECC** method is known as **SEDED** : **S**ingle **E**rror **C**orrect **D**ouble **E**rror **D**etect
 - It was developed by **R.W. Hamming** for error correction

- ❑ **There are two major components of Hamming Code error correcting system :**
 - **Encoder :**
 - At the source encodes the message by inserting redundant bits within the message
 - Redundant bits are extra bits (i.e., **parity bits**) that are generated and inserted at specific positions in the message itself to enable error detection and correction.
 - **Decoder :**
 - At the destination decodes the encoded message received, it performs recalculations of parity bit values to detect errors and find the bit position that has error in case of single bit flip
 - In case of double bit errors, does not correct any bit flip, instead detects and notify to the system hardware and/or to software infrastructure to take further action.

Encoding a message by Hamming Code

- ❑ **The procedure used by the sender to encode the message encompasses the following steps –**
 - Step 1 – Calculation of the number of redundant bits (i.e parity bits).
 - Step 2 – Positioning the parity bits within the message bits
 - Step 3 – Calculating the values of each parity bits

- ❑ **Note : In remainder of the slides redundant bits will be referenced as parity bits**

Encoding Step-1 : Calculation of the number of parity bits

- ❑ If the message contains M number of data bits, R number of parity bits are added to it so that it can indicate at least $(M + R + 1)$ different states.
 - $(M + R)$ indicates location of an error in each of bit position
 - $+1$ reference above indicates no error state
- Since, R bits can indicate 2^R states, 2^R must be at least equal to $(M + R + 1)$. Thus, the following equation should hold :
 - $2^R \geq M + R + 1$
- Example(s) of number of parity bits for a given size of message data :
 - If the message data is of 8 bits, i.e., $M = 8$, the minimum value of R that will satisfy the above equation is 4, ($2^4 \geq 8 + 4 + 1$). The total number of bits in the encoded message, $(M + R) = 12$. This is referred as (12,4) Hamming code
 - If the message data is of 15 bits, i.e., $M = 15$, the minimum value of R that will satisfy the above equation is 5, ($2^5 \geq 15 + 5 + 1$). The total number of bits in the encoded message, $(M + R) = 20$. This is referred as (20,5) Hamming code

Encoding Step-2 : Positioning the parity bits

❑ The parity bits are placed at the bit positions of powers of 2, i.e., 1, 2, 4, 8, 16 etc.

- These parity bits are referred as p0 (at position $2^0=1$), p1 (at position $2^1=2$), p2(at position $2^2=4$), p3 (at position $2^3=8$) and so on.

▪ **Example :**

- If $M = 8$ and $R = 4$, the parity bits (p0, p1, p2, p3) will be inserted in power of 2 positions (1, 2, 4, 8) within message bits (d0, d1, d2, d3, d4, d5, d6, d7) as shown below :

Bit Position →	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC Code →	d7	d6	d5	d4	p3	d3	d2	d1	p2	d0	p1	p0	op

• **Note(s):**

- **ECC Code = Message data bits + Parity (redundant) bits + Overall Parity Bit**
- Bit position '0' in ECC code shown above is reserved for overall parity (**op**). Also known as global parity. In this bit position neither message data bits or parity p0,p1,p2,p3 parity bits cannot be placed
- Message data bits **d0** to **d7** will not be placed in bit positions 1,2,4,8 as all bit positions which are power of two 2^n these are reserved for parity bits
- All non power of two-bit positions are for data bits. First message data bit **d0** will be placed in very first non-power of 2 position which is bit position '3' as shown above
- Second message data bit **d1** will be placed in sequence after d0 in the very next non-power of 2 position which is bit position '5' as shown above
- Similarly, remainder of message data bits, d2, d3, d4, d5, d6 will be placed in bit positions 6,7,9,10,11,12

Encoding Step-3 : Calculating the values of each parity bit

❑ Each of the parity bits can be either even or odd parity. Here on let us consider even parity !

❑ Each parity bit in hamming code, is calculated based upon its bit position. For e.g., M=8, R=4:

- First create a binary value table for 1 to (M+R) value as shown below. Number bits as : 1,2,3,5,6,7 and so on and its equivalent binary value. If R=4, then there will be 4 columns and binary value will be represented as 4-bit values
- All bit positions that are powers of two (have a single 1 bit in the binary form of their position) are parity bits: 1, 2, 4, 8, etc. (1, 10, 100, 1000)
- All other bit positions, with two or more 1 bits in the binary form of their position, are data bits.
- Parity bits cover all data bit positions whose binary representation includes a 1 in the i^{th} position except the ones which are reserved for parity bit itself. Thus -
 - **p0** is the parity bit for all data bits in positions whose binary representation includes a 1 in the least significant position excluding 1 (bit position 3, 5, 7, 9, 11 and so on)
$$p0 = d0 \wedge d1 \wedge d3 \wedge d4 \wedge d6$$
 - **p1** is the parity bit for all data bits in positions whose binary representation includes a 1 in the position 2 from right except 2 (bit position 3, 6, 7, 10, 11 and so on)
$$p1 = d0 \wedge d2 \wedge d3 \wedge d5 \wedge d6$$

Binary value table M=8, R=4 (M+R=12)

Bit Position	p3	p2	p1	p0	Inference
1	0	0	0	1	p0 bit error
2	0	0	1	0	p1 bit error
3	0	0	1	1	d0 bit error
4	0	1	0	0	p2 bit error
5	0	1	0	1	d1 bit error
6	0	1	1	0	d2 bit error
7	0	1	1	1	d3 bit error
8	1	0	0	0	p3 bit error
9	1	0	0	1	d4 bit error
10	1	0	1	0	d5 bit error
11	1	0	1	1	d6 bit error
12	1	1	0	0	d7 bit error

Encoding Step-3 : Calculating the values of each parity bit

- **p2** is the parity bit for all data bits in positions whose binary representation includes a 1 in the position 3 from right except 4 (bit position 5, 6, 7, 12 and so on)

$$p2 = d1 \wedge d2 \wedge d3 \wedge d7$$

- **p3** is the parity bit for all data bits in positions whose binary representation includes a 1 in the position 4 from right except 8 (bit position 9, 10, 11, 12 and so on)

$$p3 = d4 \wedge d5 \wedge d6 \wedge d7$$

This general rule for parity bits can be shown visually. See table below

Source : https://en.wikipedia.org/wiki/Hamming_code

Bit position		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	
Encoded data bits		p0	p1	d0	p2	d1	d2	d3	p3	d4	d5	d6	d7	d8	d9	d10	p4	d11	d12	d13	d14	
Parity bit coverage	p0	✓		✓		✓		✓		✓		✓		✓		✓		✓		✓		
	p1		✓	✓			✓	✓			✓	✓			✓	✓			✓	✓		...
	p2				✓	✓	✓	✓					✓	✓	✓	✓					✓	
	p3								✓	✓	✓	✓	✓	✓	✓	✓						
	p4																✓	✓	✓	✓	✓	

For M+R=12 see above parity bit coverage

IMP NOTE : As one can see that same data bit is covered by multiple parity bits. Example : p0 and p1 cover d0, d3 and d6 data bits. Similarly, p1 and p2 cover d2 and d3 data bits and so on.

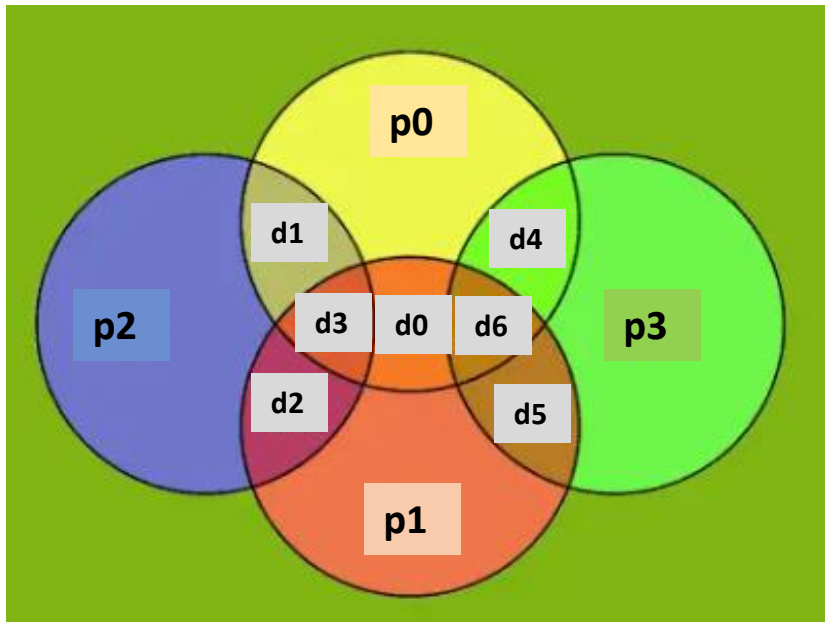
This overlapping parity coverage, helps locate the position of data bit in case of single bit error

Binary value table M=8, R=4 (M+R=12)

Bit Position	p3	p2	p1	p0	Inference
1	0	0	0	1	p0 bit error
2	0	0	1	0	p1 bit error
3	0	0	1	1	d0 bit error
4	0	1	0	0	p2 bit error
5	0	1	0	1	d1 bit error
6	0	1	1	0	d2 bit error
7	0	1	1	1	d3 bit error
8	1	0	0	0	p3 bit error
9	1	0	0	1	d4 bit error
10	1	0	1	0	d5 bit error
11	1	0	1	1	d6 bit error
12	1	1	0	0	d7 bit error

Parity Coverage View

- ❑ As once can see that same data bit is covered by multiple parity bits. Example : p0 and p1 cover d0 and d3 data bits. Similarly, p2 and p3 cover d2 and d3 data bits and so on.
- ❑ This overlapping parity coverage, helps locate the position of data bit in case of single bit error



Binary value table M=8, R=4 (M+R=12)

Bit Position	p3	p2	p1	p0	Inference
1	0	0	0	1	p0 bit error
2	0	0	1	0	p1 bit error
3	0	0	1	1	d0 bit error
4	0	1	0	0	p2 bit error
5	0	1	0	1	d1 bit error
6	0	1	1	0	d2 bit error
7	0	1	1	1	d3 bit error
8	1	0	0	0	p3 bit error
9	1	0	0	1	d4 bit error
10	1	0	1	0	d5 bit error
11	1	0	1	1	d6 bit error
12	1	1	0	0	d7 bit error

Note : p2 and P3 covers d7 which is not shown above in the diagram

Encoding Step-3 : Calculating the values of each parity bit

❑ Example : Lets assume original message data bits are : **11001110** which needs to be encoded using even parity hamming code.

- In this example, $M=8$ and using $2^R \geq M + R + 1$, number of redundant (parity) bits $R = 4$
- Message bits : 11001110 (d0=0 is LSB and d7=1 is MSB)
- This is how message data bits will be placed in encoded message along with parity bits :

Bit Position →	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC Code →	d7=1	d6=1	d5=0	d4=0	p3	d3=1	d2=1	d1=1	p2	d0=0	p1	p0	op

❑ Let's compute p0, p1, p2, p3 parity bits (even parity) using below mentioned formula :

- **p0** = $d0 \wedge d1 \wedge d3 \wedge d4 \wedge d6 = 0 \wedge 1 \wedge 1 \wedge 0 \wedge 1 = 1$
- **p1** = $d0 \wedge d2 \wedge d3 \wedge d5 \wedge d6 = 0 \wedge 1 \wedge 1 \wedge 0 \wedge 1 = 1$
- **p2** = $d1 \wedge d2 \wedge d3 \wedge d7 = 1 \wedge 1 \wedge 1 \wedge 1 = 0$
- **p3** = $d4 \wedge d5 \wedge d6 \wedge d7 = 0 \wedge 0 \wedge 1 \wedge 1 = 0$

❑ ECC Code with parity bits infused within message bits : **110001110011**

❑ Finally, compute overall even parity = $1 \wedge 1 \wedge 0 \wedge 0 \wedge 0 \wedge 1 \wedge 1 \wedge 1 \wedge 0 \wedge 0 \wedge 1 \wedge 1 = 1$

Bit Position →	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC Code →	d7=1	d6=1	d5=0	d4=0	p3=0	d3=1	d2=1	d1=1	p2=0	d0=0	p1=1	p0=1	op=1

Decoding a message in Hamming Code

- ❑ **Once the receiver receives an encoded incoming message, it performs recalculations to detect errors and correct them. The steps for recalculation are –**
 - Step 1 – Re-compute Parity values
 - Step 2 – Error detection and correction

Decoding Step-1 : Re-compute parity values

❑ Once the message is received at the receiver, ECC decoder will re-compute parity bits using similar technique which was employed by the encoder :

- $p0$ = XOR data bits at location (3, 5, 7, 9, 11 and so on)
$$p0 = d0 \wedge d1 \wedge d3 \wedge d4 \wedge d6$$
- $p1$ = XOR data bits at location (3, 6, 7, 10, 11 and so on)
$$p1 = d0 \wedge d2 \wedge d3 \wedge d5 \wedge d6$$
- $p2$ = XOR data bits at location (5, 6, 7, 12 and so on)
$$p2 = d1 \wedge d2 \wedge d3 \wedge d7$$
- $p3$ = XOR data bits at location (9, 10, 11, 12 and so on)
$$p3 = d4 \wedge d5 \wedge d6 \wedge d7$$

❑ Re-compute overall parity for incoming encoded message bits using XOR function

❑ Note :

- Decoder at the receiver, also has the understanding that even parity hamming code scheme was employed by encoder at the transmitter and accordingly **ECC** code was generated
- Decoder also computes number of parity (i.e., redundant bits) using $2^R \geq M + R + 1$ equation
- Decoder since it knows the size of the message, it also knows which specific bits in incoming encoded message are parity bits and message bits

Decoding Step-2 : Error detection and correction

❑ Generate syndrome value which indicates specific bit that has error

- Syndrome value is computed by comparing parity bits in incoming encoded message with re-computed parity bit at the receiver by decoder
- If original parity bits in incoming message are p_3, p_2, p_1, p_0 and re-computed parity bits are c_3, c_2, c_1, c_0 then syndrome value can be generated using xor :
 - $\text{Syndrome}[3] = p_3 \wedge c_3$
 - $\text{Syndrome}[2] = p_2 \wedge c_2$
 - $\text{Syndrome}[1] = p_1 \wedge c_1$
 - $\text{Syndrome}[0] = p_0 \wedge c_0$

❑ Then, compare overall parity bit in incoming encoded message with re-computed at the receiver

#	Syndrome Value	Comparison Result of incoming Overall Parity with Re-computed Overall Parity	Conclusion / Inference
1	Syndrome == 0	Match	No Error scenario. Hence no error correction required.
2	Syndrome == 0	Mis-Match	Error has occurred in overall parity "op" bit. So, the message data "d" bits can be safely consumed by the receiver after removing parity and overall parity bits
3	Syndrome != 0	Mis-Match	Single bit error occurred and can be corrected by reversing the bit value at the bit position given by syndrome value. Single bit error could be either in one of the message 'd' bits or in one of the parity bit's (p_0 or p_1 or p_2 or p_3)
4	Syndrome != 0	Match	Double bit error detected which cannot be corrected. Message data bits not safe to consume by receiver. Message data bits should be discarded since uncorrectable

Decoding Example-1 : Single bit error detection and correction case

❑ Example-1 : Single bit error detection and correction case

- Let's assume, transmitter after encoding sends to receiver encoded message 1100011100111
- During transmission of the encoded message one of the data bits gets flipped: 1000011100111
- Receiver receives encoded message with single bit error
- See below, incoming encoded message with d6 data bit flipped to 0 :

Bit Position →	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC Code Received	d7=1	d6=0	d5=0	d4=0	p3=0	d3=1	d2=1	d1=1	p2=0	d0=0	p1=1	p0=1	op=1

❑ Re-computed parity bits by decoder :

- $c0 = d0 \wedge d1 \wedge d3 \wedge d4 \wedge d6 = 0 \wedge 1 \wedge 1 \wedge 0 \wedge 0 = 0$
- $c1 = d0 \wedge d2 \wedge d3 \wedge d5 \wedge d6 = 0 \wedge 1 \wedge 1 \wedge 0 \wedge 0 = 0$
- $c2 = d1 \wedge d2 \wedge d3 \wedge d7 = 1 \wedge 1 \wedge 1 \wedge 1 = 0$
- $c3 = d4 \wedge d5 \wedge d6 \wedge d7 = 0 \wedge 0 \wedge 0 \wedge 1 = 1$
- **Overall Parity (op) = $1 \wedge 0 \wedge 0 \wedge 0 \wedge 0 \wedge 1 \wedge 1 \wedge 1 \wedge 0 \wedge 0 \wedge 1 \wedge 1 = 0$ (computed using incoming message with original parity value and removing original overall parity removed)**
- Replacing original parity bits in encoded message received by computed parity bits (just for visualization)

Bit Position →	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC Code after parity re-computed	d7=1	d6=0	d5=0	d4=0	c3=1	d3=1	d2=1	d1=1	c2=0	d0=0	c1=0	c0=0	op=0

Decoding Example-1 : Single bit error detection and correction case

❑ Compute Syndrome value to detect error and identify data bit which has the error :

- Original parity bits in received message compared with re-computed parity bits
- Original overall parity bit in received message compared with re-computed overall parity bit

Bit Position →	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC Code Received	d7=1	d6=0	d5=0	d4=0	p3=0	d3=1	d2=1	d1=1	p2=0	d0=0	p1=1	p0=1	op=1
					≠				=		≠	≠	≠
Bit Position →	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC Code after parity re-computed	d7=1	d6=0	d5=0	d4=0	c3=1	d3=1	d2=1	d1=1	c2=0	d0=0	c1=0	c0=0	op=0

❑ Original parity bits in incoming encoded message : $p3 = 0$, $p2 = 0$, $p1 = 1$, $p0 = 1$

❑ Re-computed parity at the receiver : $c3 = 1$, $c2 = 0$, $c1 = 0$, $c0 = 0$

❑ Comparing original parity with re-computed parity bits to generate **Syndrome Value** :

- $p3 \wedge c3 = 0 \wedge 1 = 1$
- $p2 \wedge c2 = 0 \wedge 0 = 0$
- $p1 \wedge c1 = 1 \wedge 0 = 1$
- $p0 \wedge c0 = 1 \wedge 0 = 1$

Since Syndrome value = 1011 which is $\neq 0000$ and overall parity bit in incoming message and re-computed overall parity by decoder does not match. This indicates single bit error.

Syndrome value=1011 which is decimal value=11, this value indicates the bit position in incoming encoded message which was flipped. In above mentioned example, 11th bit position has d6 message bit which was flipped to 0. Now, decoder can safely correct this by flipping it back to '1'

❑ Overall parity received does not match re-computed overall parity : $1 \neq 0$

Decoding Example-2 : Overall parity bit error detection case

❑ Example-2 : Overall parity bit error detection case

- Let's assume, transmitter after encoding sends to receiver encoded message 1100011100111
- During transmission of the encoded message overall parity bit gets flipped: 110001110011**0**
- Receiver receives encoded message with single bit error
- See below, incoming encoded message with overall parity bit (**op**) flipped to 0 :

Bit Position →	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC Code Received →	d7=1	d6=1	d5=0	d4=0	p3=0	d3=1	d2=1	d1=1	p2=0	d0=0	p1=1	p0=1	op=0

❑ Re-computed parity bits by decoder :

- $c0 = d0 \wedge d1 \wedge d3 \wedge d4 \wedge d6 = 0 \wedge 1 \wedge 1 \wedge 0 \wedge 1 = 1$
- $c1 = d0 \wedge d2 \wedge d3 \wedge d5 \wedge d6 = 0 \wedge 1 \wedge 1 \wedge 0 \wedge 1 = 1$
- $c2 = d1 \wedge d2 \wedge d3 \wedge d7 = 1 \wedge 1 \wedge 1 \wedge 1 = 0$
- $c3 = d4 \wedge d5 \wedge d6 \wedge d7 = 0 \wedge 0 \wedge 1 \wedge 1 = 0$
- Overall Parity op bit** = $1 \wedge 1 \wedge 0 \wedge 0 \wedge 0 \wedge 1 \wedge 1 \wedge 1 \wedge 0 \wedge 0 \wedge 1 \wedge 1 = 1$ (computed using incoming message with original parity value and removing original overall parity removed)
- Replacing original overall parity bits in encoded message received by computed overall parity value

Bit Position →	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC Code after parity re-computed →	d7=1	d6=1	d5=0	d4=0	c3=0	d3=1	d2=1	d1=1	c2=0	d0=0	c1=1	c0=1	op=1

Decoding Example-2 : Overall parity bit detection case

❑ Compute Syndrome value to detect error and identify data bit which has the error :

- Original parity bits in received message compared with re-computed parity bits
- Original overall parity bit in received message compared with re-computed overall parity bit

Bit Position →	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC Code Received	d7=1	d6=1	d5=0	d4=0	p3=0	d3=1	d2=1	d1=1	p2=0	d0=0	p1=1	p0=1	op=0
					=				=		=	=	≠
Bit Position →	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC Code after parity re-computed	d7=1	d6=1	d5=0	d4=0	c3=0	d3=1	d2=1	d1=1	c2=0	d0=0	c1=1	c0=1	op=1

❑ Original parity bits in incoming encoded message : $p3 = 0$, $p2 = 0$, $p1 = 1$, $p0 = 1$

❑ Re-computed parity at the receiver : $c3 = 0$, $c2 = 0$, $c1 = 1$, $c0 = 1$

❑ Comparing original parity with re-computed parity bits to generate **Syndrome Value** :

▪ $p3 \wedge c3 = 0 \wedge 0 = 0$

▪ $p2 \wedge c2 = 0 \wedge 0 = 0$

▪ $p1 \wedge c1 = 1 \wedge 1 = 0$

▪ $p0 \wedge c0 = 1 \wedge 1 = 0$

Since Syndrome value == 0000 and overall parity in incoming encoded message does not match with re-computed overall parity, this indicates overall parity bit in incoming message had error. Receiver can now safely use message 'd' bits after dis-carding all parity bits

❑ Overall parity received does not match re-computed overall parity : $0 \neq 1$

Decoding Example-3 : No Error case

❑ Example-3 : No error case

- Let's assume, transmitter after encoding sends to receiver encoded message 1100011100111
- Receiver receives encoded message without any errors : 1100011100111
- See below, incoming encoded message :

Bit Position →	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC Code Received →	d7=1	d6=1	d5=0	d4=0	p3=0	d3=1	d2=1	d1=1	p2=0	d0=0	p1=1	p0=1	op=1

❑ Re-computed parity bits by decoder :

- $c0 = d0 \wedge d1 \wedge d3 \wedge d4 \wedge d6 = 0 \wedge 1 \wedge 1 \wedge 0 \wedge 1 = 1$
- $c1 = d0 \wedge d2 \wedge d3 \wedge d5 \wedge d6 = 0 \wedge 1 \wedge 1 \wedge 0 \wedge 1 = 1$
- $c2 = d1 \wedge d2 \wedge d3 \wedge d7 = 1 \wedge 1 \wedge 1 \wedge 1 = 0$
- $c3 = d4 \wedge d5 \wedge d6 \wedge d7 = 0 \wedge 0 \wedge 1 \wedge 1 = 0$
- **Overall Parity op bit** = $1 \wedge 1 \wedge 0 \wedge 0 \wedge 0 \wedge 1 \wedge 1 \wedge 1 \wedge 0 \wedge 0 \wedge 1 \wedge 1 = 1$ (computed using incoming message with original parity value and removing original overall parity removed)
- Replacing original parity bits in encoded message received by computed parity bits(just for visualization)

Bit Position →	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC Code after parity re-computed →	d7=1	d6=1	d5=0	d4=0	c3=0	d3=1	d2=1	d1=1	c2=0	d0=0	c1=1	c0=1	op=1

Decoding Example-3 : No error case

❑ Compute Syndrome value to detect error and identify data bit which has the error :

- Original parity bits in received message compared with re-computed parity bits
- Original overall parity bit in received message compared with re-computed overall parity bit

Bit Position →	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC Code Received	d7=1	d6=1	d5=0	d4=0	p3=0	d3=1	d2=1	d1=1	p2=0	d0=0	p1=1	p0=1	op=1
					=				=		=	=	=
Bit Position →	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC Code after parity re-computed	d7=1	d6=1	d5=0	d4=0	c3=0	d3=1	d2=1	d1=1	c2=0	d0=0	c1=1	c0=1	op=1

❑ Original parity bits in incoming encoded message : $p3 = 0$, $p2 = 0$, $p1 = 1$, $p0 = 1$

❑ Re-computed parity at the receiver : $c3 = 0$, $c2 = 0$, $c1 = 1$, $c0 = 1$

❑ Comparing original parity with re-computed parity bits to generate **Syndrome Value** :

- $p3 \wedge c3 = 0 \wedge 0 = 0$
- $p2 \wedge c2 = 0 \wedge 0 = 0$
- $p1 \wedge c1 = 1 \wedge 1 = 0$
- $p0 \wedge c0 = 1 \wedge 1 = 0$

Since Syndrome value == 0000 and overall parity in incoming encoded message does match with re-computed overall parity, this indicates there is no error. Receiver can now safely use message 'd' bits after dis-carding all parity bits

❑ Overall parity received matched with re-computed overall parity : $1 == 1$

Decoding Example-4 : Double bit error case

Example-4 : Double bit error case

- Let's assume, transmitter after encoding sends to receiver encoded message 1100011100111
- Receiver receives encoded message with double bit errors : 0000011100111
- See below, incoming encoded message with d7 and d6 bit flipped to value 0 from 1:

Bit Position →	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC Code Received	d7=0	d6=0	d5=0	d4=0	p3=0	d3=1	d2=1	d1=1	p2=0	d0=0	p1=1	p0=1	op=1

Re-computed parity bits by decoder :

- $c0 = d0 \wedge d1 \wedge d3 \wedge d4 \wedge d6 = 0 \wedge 1 \wedge 1 \wedge 0 \wedge 0 = 0$
- $c1 = d0 \wedge d2 \wedge d3 \wedge d5 \wedge d6 = 0 \wedge 1 \wedge 1 \wedge 0 \wedge 0 = 0$
- $c2 = d1 \wedge d2 \wedge d3 \wedge d7 = 1 \wedge 1 \wedge 1 \wedge 0 = 1$
- $c3 = d4 \wedge d5 \wedge d6 \wedge d7 = 0 \wedge 0 \wedge 0 \wedge 0 = 0$
- Overall Parity op bit = $0 \wedge 0 \wedge 0 \wedge 0 \wedge 0 \wedge 1 \wedge 1 \wedge 1 \wedge 0 \wedge 0 \wedge 1 \wedge 1 = 1$ (computed using incoming message with original parity value and removing original overall parity removed)**
- Replacing original parity bits in encoded message received by computed parity bits(just for visualization)

Bit Position →	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC Code after parity re-computed	d7=0	d6=0	d5=0	d4=0	c3=0	d3=1	d2=1	d1=1	c2=1	d0=0	c1=0	c0=0	op=1

Decoding Example-4 : Double bit error detection case

❑ Compute Syndrome value to detect error and identify data bit which has the error :

- Original parity bits in received message compared with re-computed parity bits
- Original overall parity bit in received message compared with re-computed overall parity bit

Bit Position →	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC Code Received	d7=0	d6=0	d5=0	d4=0	p3=0	d3=1	d2=1	d1=1	p2=0	d0=0	p1=1	p0=1	op=1
					=				≠		≠	≠	=
Bit Position →	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC Code after parity re-computed	d7=0	d6=0	d5=0	d4=0	c3=0	d3=1	d2=1	d1=1	c2=1	d0=0	c1=0	c0=0	op=1

❑ Original parity bits in incoming encoded message : $p3 = 0$, $p2 = 0$, $p1 = 1$, $p0 = 1$

❑ Re-computed parity at the receiver : $c3 = 0$, $c2 = 1$, $c1 = 0$, $c0 = 0$

❑ Comparing original parity with re-computed parity bits to generate **Syndrome Value** :

- $p3 \wedge c3 = 0 \wedge 0 = 0$
- $p2 \wedge c2 = 0 \wedge 1 = 1$
- $p1 \wedge c1 = 1 \wedge 0 = 1$
- $p0 \wedge c0 = 1 \wedge 0 = 1$

Since Syndrome value = 0111 which is $\neq 0000$ and overall parity bit does match. This indicates double bit error. In this case syndrome value cannot be used to detect which specific bit has error. Hence error cannot be corrected. Receiver should discard message data bits in such cases.

❑ Overall parity received matched with re-computed overall parity : $1 == 1$

Decoding Example-5 : Parity bit error detection case

❑ Example-5 : Parity bit error detection case

- Let's assume, transmitter after encoding sends to receiver encoded message 1100011100111
- During transmission of the encoded message, parity p1 bit gets flipped: 1100011100**0**11
- Receiver receives encoded message with single bit error with parity bit flipped
- See below, incoming encoded message with parity p1 data bit flipped to 0 :

Bit Position →	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC Code Received	d7=1	d6=1	d5=0	d4=0	p3=0	d3=1	d2=1	d1=1	p2=0	d0=0	p1=0	p0=1	op=1

❑ Re-computed parity bits by decoder :

- $c0 = d0 \wedge d1 \wedge d3 \wedge d4 \wedge d6 = 0 \wedge 1 \wedge 1 \wedge 0 \wedge 1 = 1$
- $c1 = d0 \wedge d2 \wedge d3 \wedge d5 \wedge d6 = 0 \wedge 1 \wedge 1 \wedge 0 \wedge 1 = 1$
- $c2 = d1 \wedge d2 \wedge d3 \wedge d7 = 1 \wedge 1 \wedge 1 \wedge 1 = 0$
- $c3 = d4 \wedge d5 \wedge d6 \wedge d7 = 0 \wedge 0 \wedge 1 \wedge 1 = 0$
- **Overall Parity op** = $1 \wedge 1 \wedge 0 \wedge 0 \wedge 0 \wedge 1 \wedge 1 \wedge 1 \wedge 0 \wedge 0 \wedge 0 \wedge 1 = 0$ (computed using incoming message with original parity value and removing original overall parity removed)
- Replacing overall parity bit in encoded message received by computed overall parity bit (for visualization)

Bit Position →	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC Code after parity re-computed	d7=1	d6=1	d5=0	d4=0	c3=0	d3=1	d2=1	d1=1	c2=0	d0=0	c1=1	c0=1	op=0

Decoding Example-5 : Parity bit error detection case

❑ Compute Syndrome value to detect error and identify data bit which has the error :

- Original parity bits in received message compared with re-computed parity bits
- Original overall parity bit in received message compared with re-computed overall parity bit

Bit Position →	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC Code Received	d7=1	d6=1	d5=0	d4=0	p3=0	d3=1	d2=1	d1=1	p2=0	d0=0	p1=0	p0=1	op=1
					=				=		≠	=	≠
Bit Position →	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC Code after parity re-computed	d7=1	d6=1	d5=0	d4=0	c3=0	d3=1	d2=1	d1=1	c2=0	d0=0	c1=1	c0=1	op=0

❑ Original parity bits in incoming encoded message : $p3 = 0$, $p2 = 0$, $p1 = 0$, $p0 = 1$

❑ Re-computed parity at the receiver : $c3 = 0$, $c2 = 0$, $c1 = 1$, $c0 = 1$

❑ Comparing original parity with re-computed parity bits to generate **Syndrome Value** :

- $p3 \wedge c3 = 0 \wedge 0 = 0$
- $p2 \wedge c2 = 0 \wedge 0 = 0$
- $p1 \wedge c1 = 0 \wedge 1 = 1$
- $p0 \wedge c0 = 1 \wedge 1 = 0$

Since Syndrome value = 0001 which is $\neq$ 0000 and overall parity bit in incoming message and re-computed overall parity by decoder does not match. This indicates single bit error.

Syndrome value=0010 which is decimal value=2, this value indicates the bit position in incoming encoded message which was flipped. In above mentioned example, 2nd bit position has parity bit p1 bit which was flipped to 0. Now, decoder can safely consume input message 'd' bits as all 'd' bits are correct without errors

❑ Overall parity received does not match re-computed overall parity : $1 \neq 0$

Disadvantages of Hamming Code

- ❑ **The Hamming coding algorithm can only detect and correct single-bit flips.**
 - If multiple bits are determined to be incorrect, the result may be that another bit that should be accurate is modified. This may lead the data to become even more erroneous.

- ❑ **Reduces bandwidth due extra overhead of carrying ECC redundant bits for each chunk of data**
 - **Example :** 16 bit of data requires extra 5 bits of redundant bits and 1-bit for overall parity is required. So additional 6 bits stored and transmitted along with each 16-bit of original data.

Homework Assignment : ECC Encoder and ECC Decoder (SECDED)

❑ Develop SystemVerilog RTL model for ECC Encoder and Decoder

- Develop ecc encoder and ecc decoder which implements Hamming Code (12,4) : M=8, R=4, 1-bit overall parity
- ECC Encoder and Decoder should detect and correct single bit error and detect double bit error (SECDED)
- In ecc_encoder template module code within function "generate_data_with_ecc", use for loops to generate parity code and placing of parity bits within ecc code instead of hard coded implementation.
 - This is required for ecc_encoder to work for other other values of M and R beyond M=8 and R=4
- In ecc_decoder template module code within function "generate_syndrome", "extract_message_data_from_ecc_code", "generate_corrected_ecc_code" use for loops to re-compute parity, extract data from ecc code and generate syndrome value.
 - This is required for ecc_decoder to work for other values of M and R beyond M=8 and R=4
- Overall parity and all other parity bits generation should be EVEN Parity in both ecc_encoder and ecc_decoder

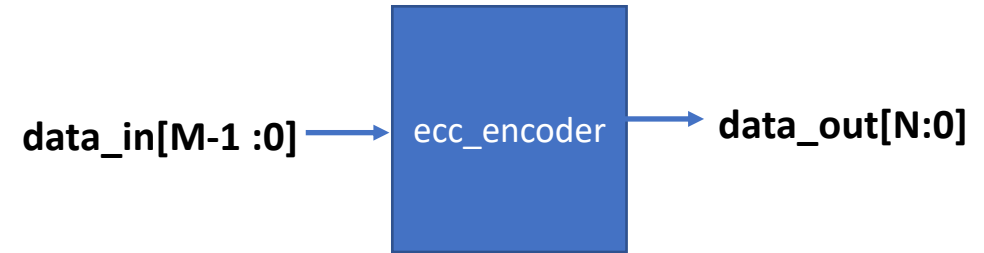
Homework Assignment : ECC Encoder and ECC Decoder (SECDED)

❑ Develop SystemVerilog RTL model for ECC Encoder and Decoder

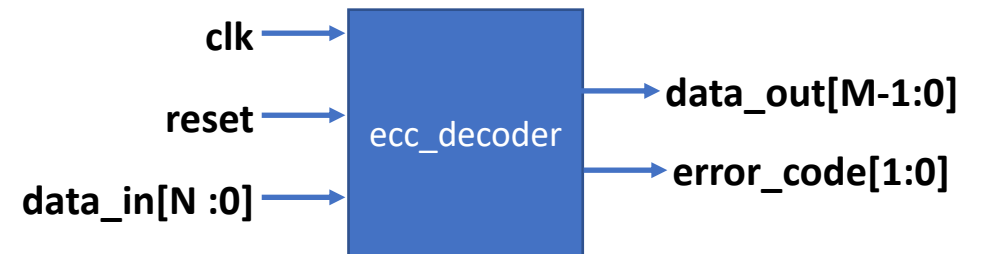
- Synthesize design and run simulation using testbench provided for M=8, R=4 implementation
- Review synthesis results (resource usage and RTL netlist/schematic) and provide snapshots of resource usage and RTL schematic for M=8, R=4
- Testbench is provided for both ecc_encoder and ecc_decoder with self checking code to indicate test pass/fail
- Testbench supports M=8 and R=4 implementation. Students to submit simulation results for M=8 and R=4.
- Students can play with testbench code and update stimulus and try other values of M and R. Simulation results in homework report submission is for M=8 and R=4. For other values, not mandatory to submit.
- Review input and output signals in simulation waveform. Provide snapshot of simulation results for M=8, R=4
- Submit in homework report :
 - Simulation waveform, Modelsim transcript snapshot which has output of \$monitor prints from testbench printed in it, Code snapshot, post synthesis RTL nestlist viewer snapshot and FPGA resource usage snapshot.

Homework Assignment-5c: ECC Encoder and ECC Decoder (SECDED)

- ❑ Assume below mentioned primary ports for **ecc_encoder** module
 - **Input data_in** : M-bit input message data bit
 - **Output data_out** : N-bit encoded message with overall parity (i.e., ECC code = Input data message bit + parity bits + overall parity bit)



- ❑ Assume below mentioned primary ports for **ecc_decoder** module
 - **Input clk** : Assume posedge clock
 - **Input reset** : Asynchronous posedge reset
 - **Input data_in** : N-bit Input encoded message data i.e., ecc_code : message data bits + parity bits + overall parity in LSB
 - **Output data_out** : M-bit output message data after removing parity bits and overall parity and after data correction
 - **Output error_code** : 2-bit output error code which indicates either single bit error, double bit error or overall parity error. Implement **error_code** as per the requirements provided in table



Syndrome Value	Comparison Result of incoming Overall Parity with Re-computed Overall Parity	error_code
syndrome == 0	Match	2'b00 : No error
syndrome == 0	Mis-Match	2'b11 : Overall parity error
syndrome != 0	Mis-Match	2'b01 : Single bit error and correct
syndrome != 0	Match	2'b10 : double bit error

References and Resources

- ❑ **Video on how to send a self-correcting message (Hamming Codes) : Must Watch !**
 - <https://www.youtube.com/watch?v=X8jsijhIIIa>

- ❑ **Hamming Code explanation in detail :**
 - https://en.wikipedia.org/wiki/Hamming_code

- ❑ **Hamming code algorithm for SECDED : (short explanation)**
 - <https://www.tutorialspoint.com/hamming-code-for-single-error-correction-double-error-detection>